ENSIAS

Software engineering

DEVSECOPS Project Presentation

AI-Assisted DevSecOps integration for an Intelligent stock market monitoring platform

SUPERVISED BY

Pr. SADKI Souad

Prepared by:

- **AIT TALEB Saad**
- **ETTALBI Omar**
- **AKOUR Ayoub**

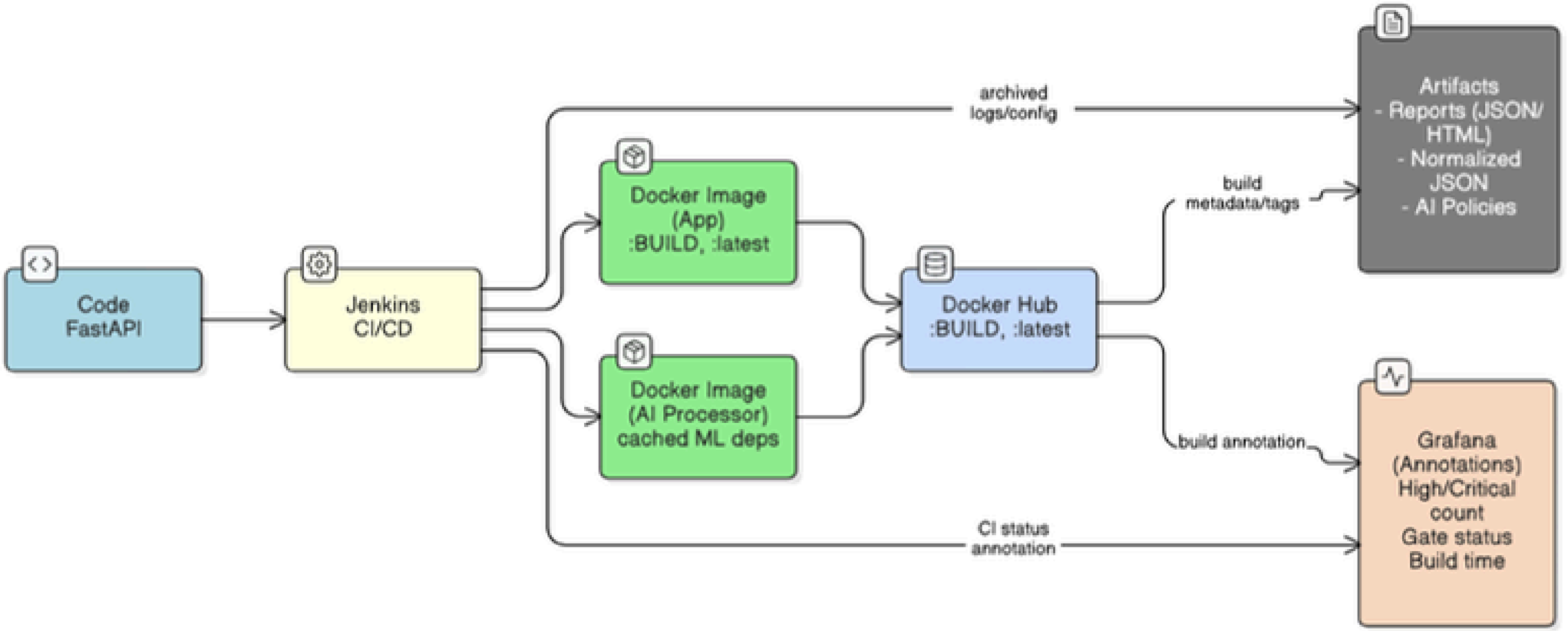
Date : 11/07/2025

OUTLINES

1. *DevOps Foundations & Core Tooling*
2. *Security Testing Pipeline*
3. *Normalization & Governance Bridge*
4. *AI-Assisted Security*
5. *Results, Traceability & Demo Path*
5. *Perspectives & Next Steps*

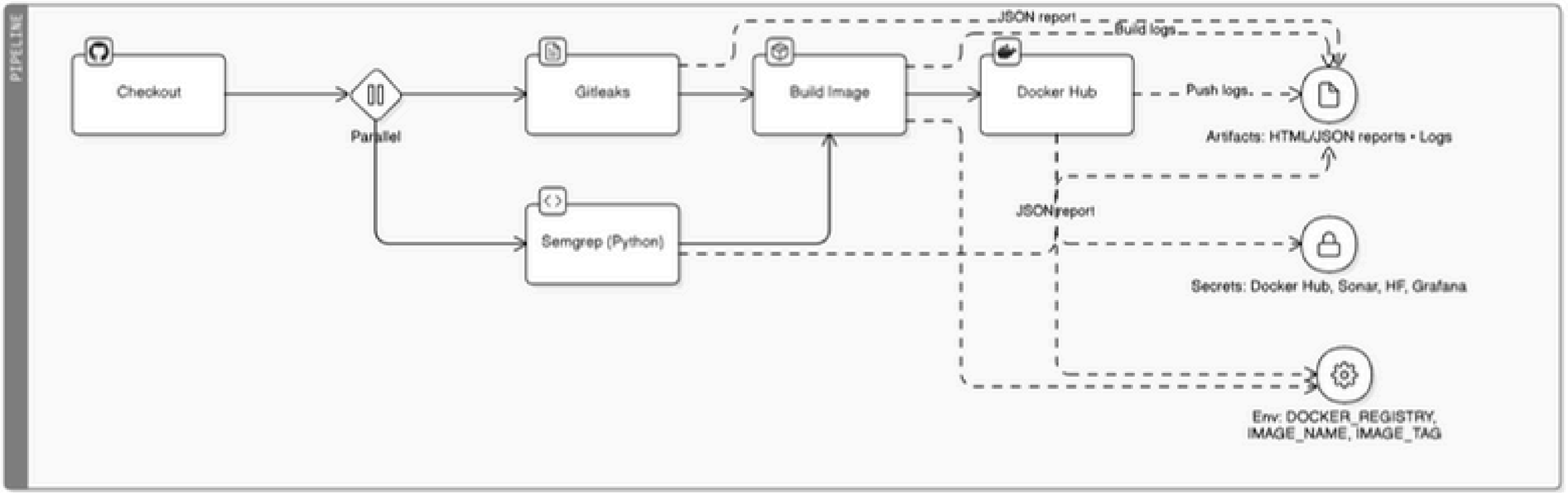
01. DevOps Foundations & Core Tooling

Code → Jenkins → Docker Images → Docker Hub → Grafana Annotations



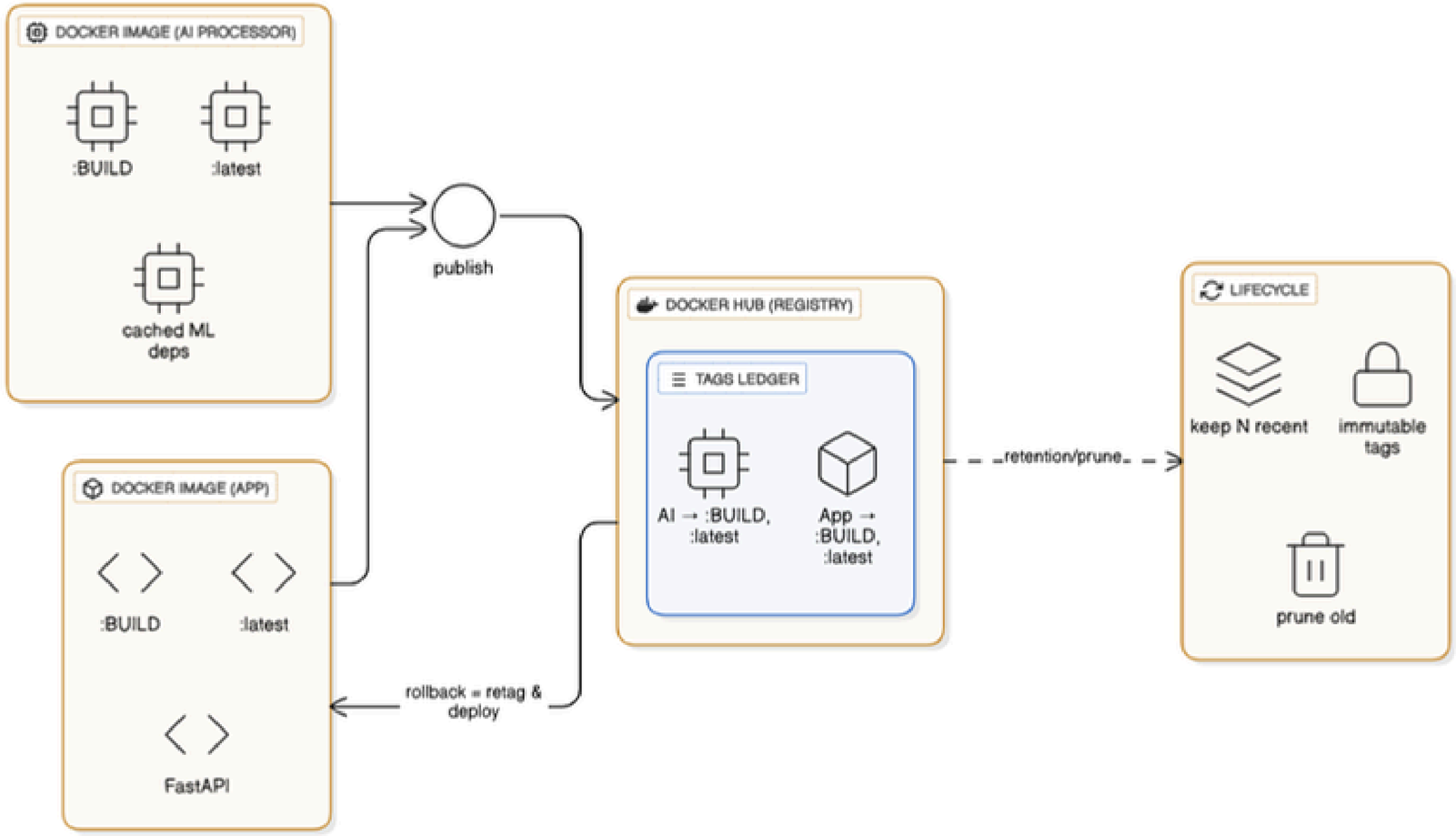
CI/CD (Jenkins)

- Parallel stages
- Secure credentials
- Deterministic tags



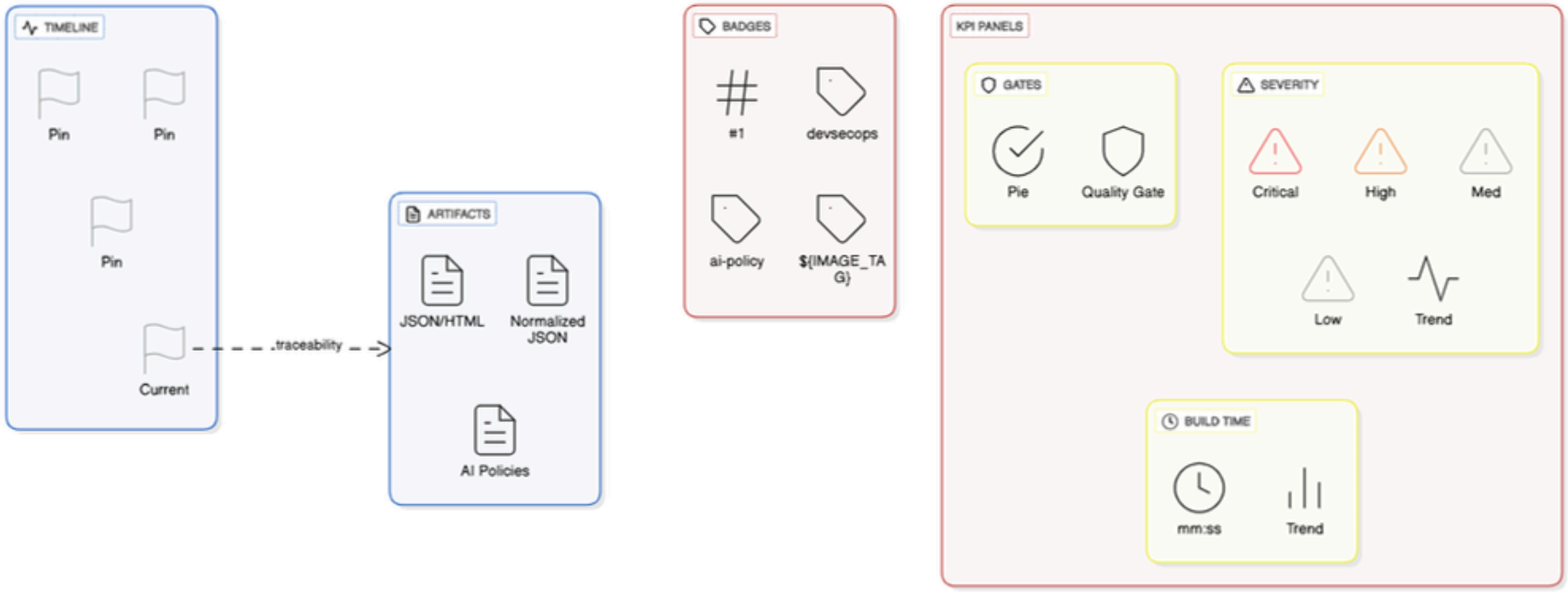
Containers & Registry

- App image
- AI processor image
- Fast rollback



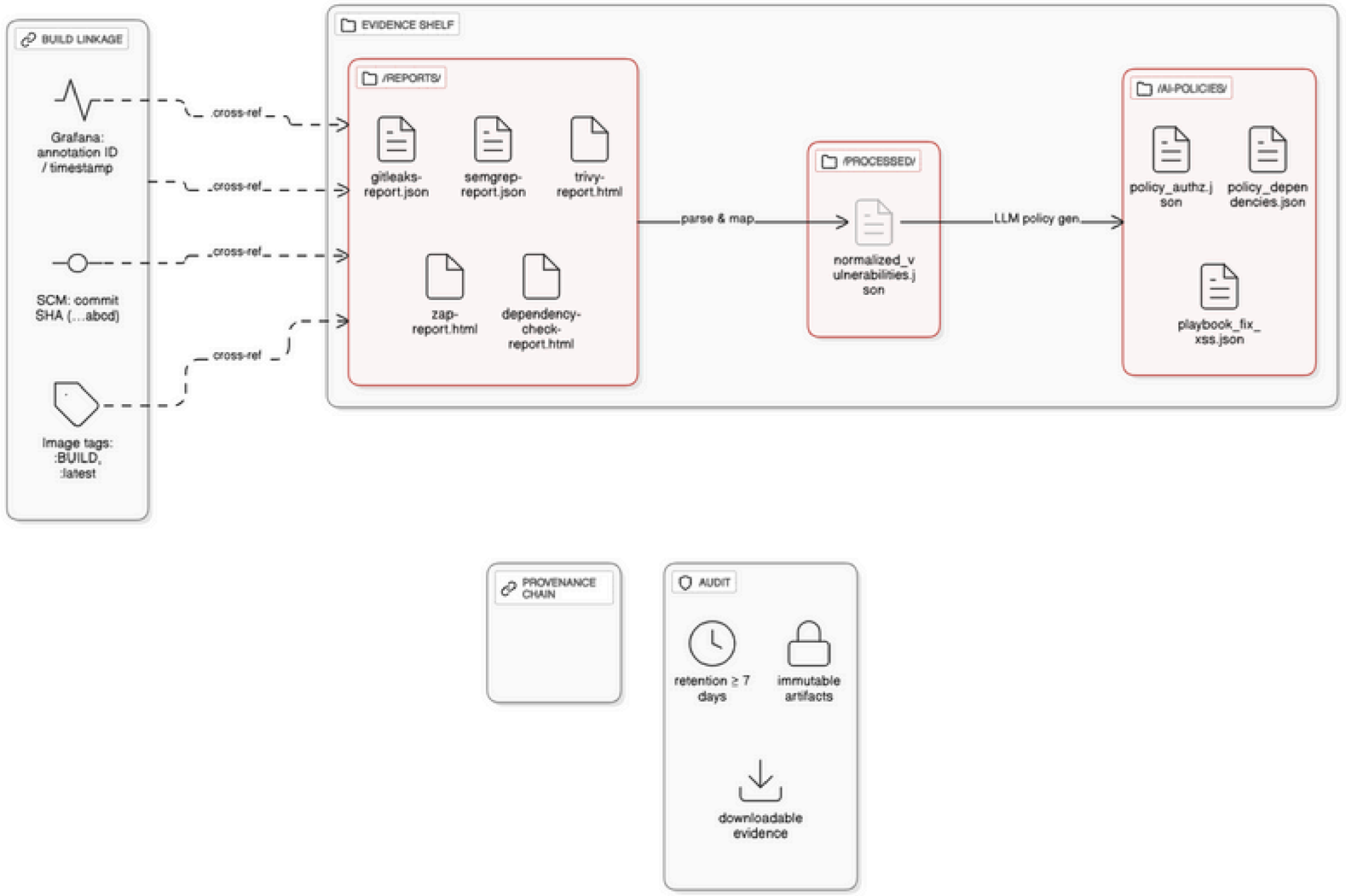
Observability

- Build annotations
- Severity counts
- Gate status



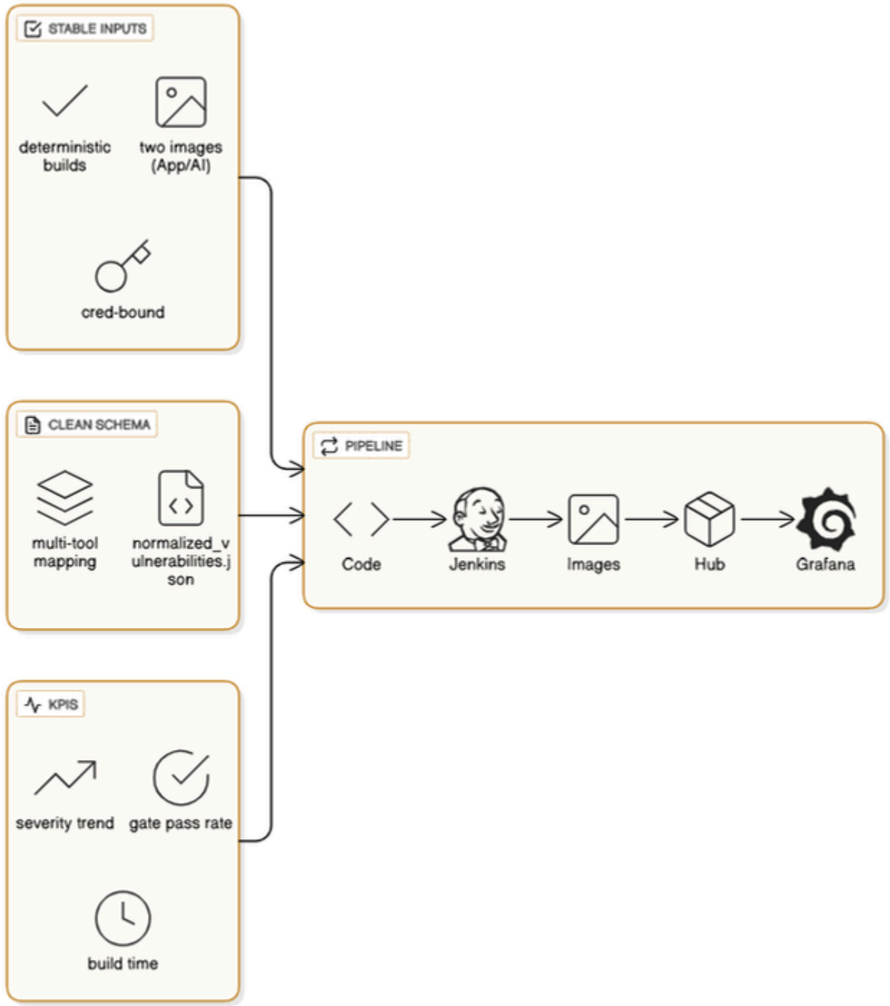
Artifacts & Traceability

- Reports archived
- Normalized JSON
- AI policies

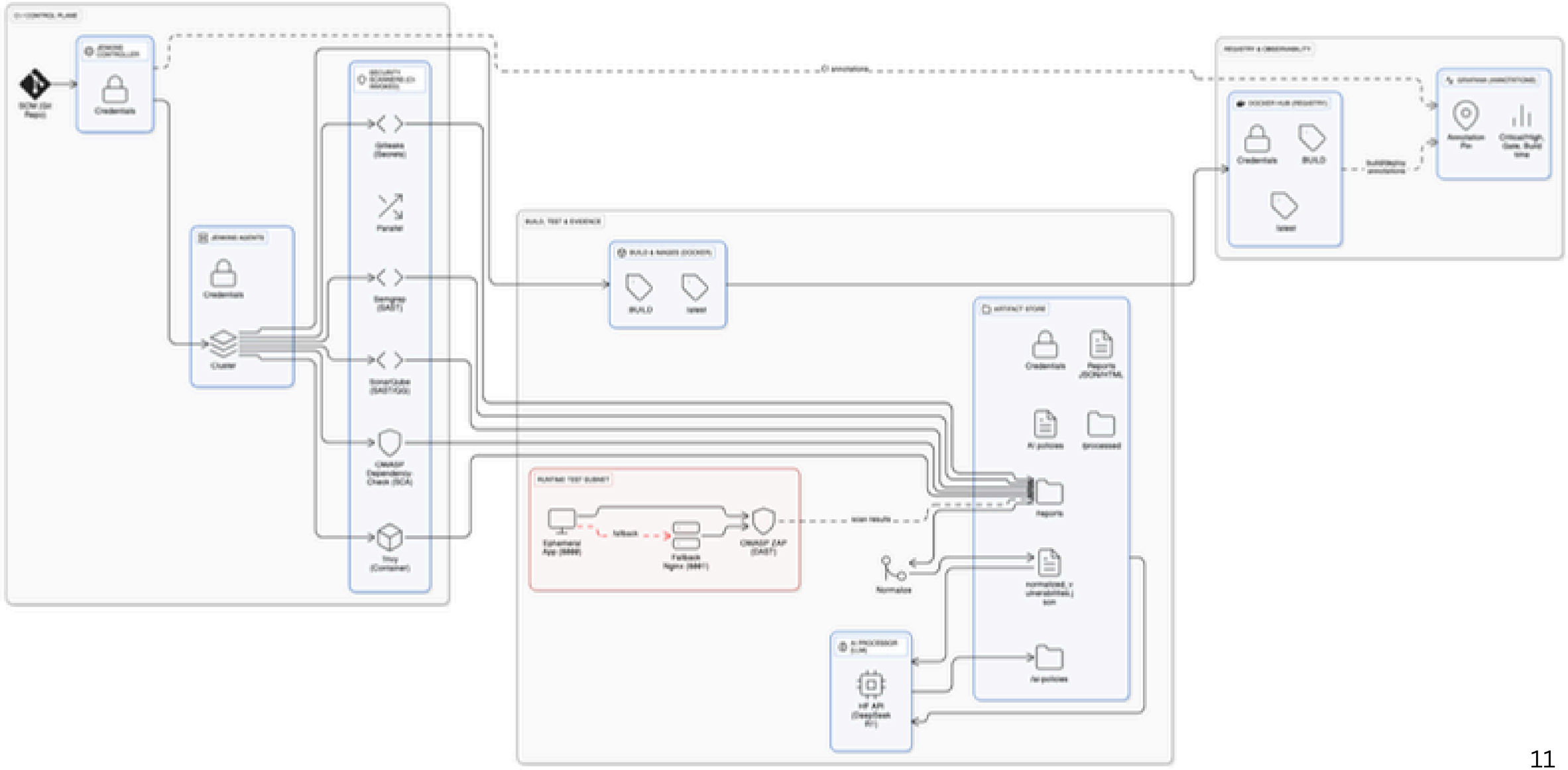


Why This Baseline

- Stable inputs → better scans
- Clean schema → better AI
- KPIs → better decisions



Architecture Overview


Credentials
Cluster
Parser
Credentials
BUILD
latest
Credentials
Reports
AI policies
Ignoreased
Reports
Normalize
Credentials
BUILD
latest

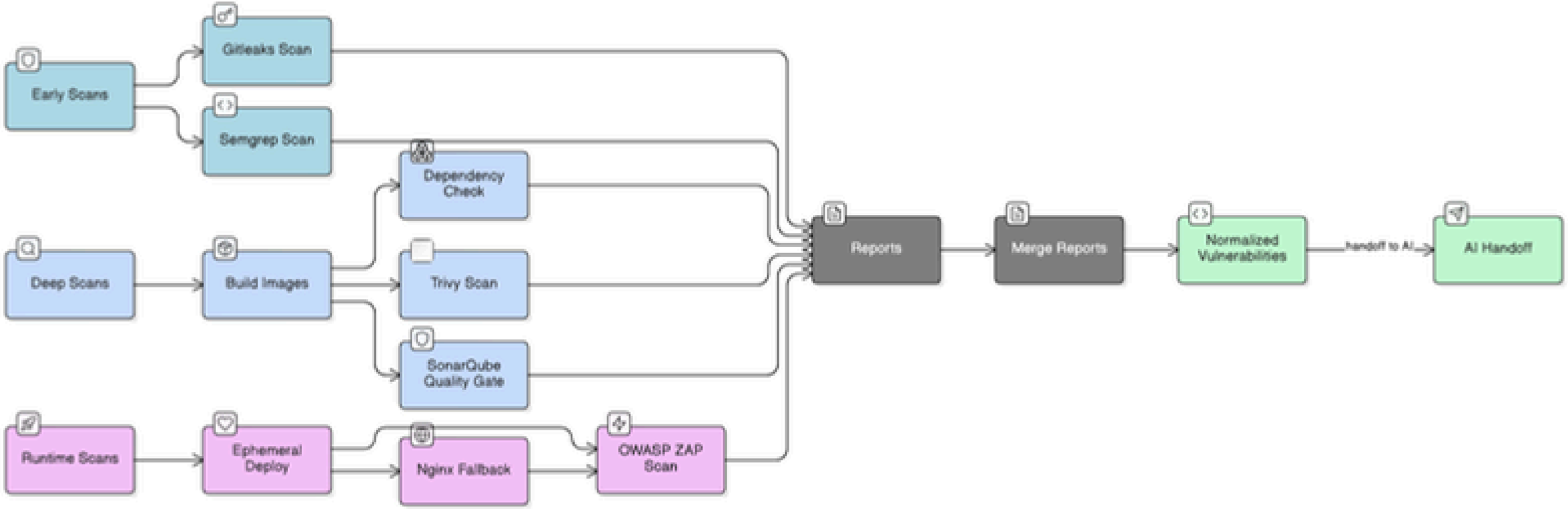
Tool purpose

Category	Tool	Purpose
Secrets Scanning	Gitleaks	Detect exposed credentials in code
SAST	Semgrep, SonarQube	Static code analysis
SCA	OWASP Dependency-Check	Find vulnerable dependencies
Container Security	Trivy	Scan Docker images for CVEs
DAST	OWASP ZAP	Dynamic web application testing
AI Analysis	HuggingFace (DeepSeek R1)	Policy generation from vulnerabilities
CI/CD	Jenkins	Pipeline automation
Monitoring	Grafana	Security metrics visualization

02. Security Testing Pipeline

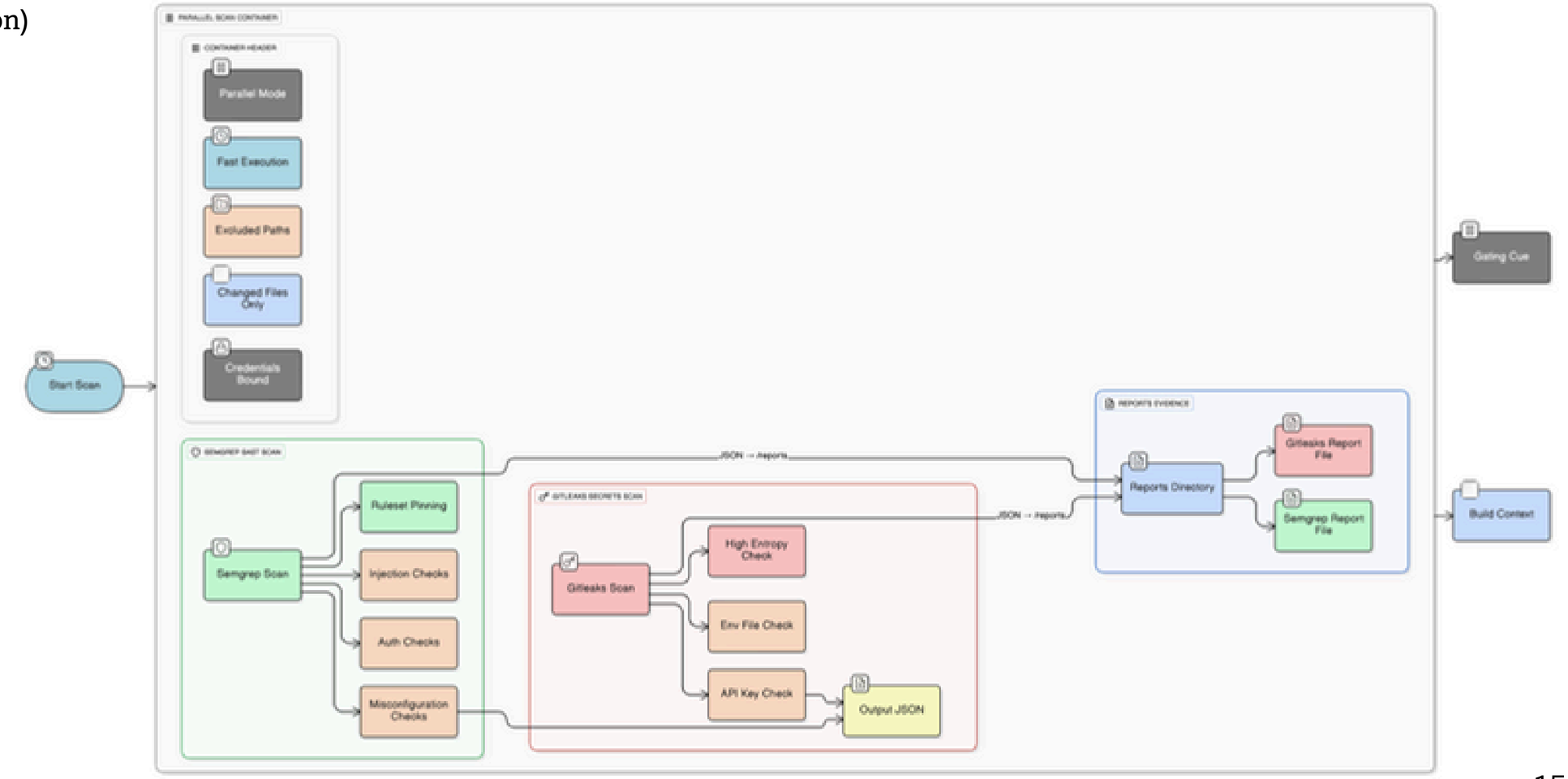
Security Pipeline Overview (Shift-Left → Runtime)

- Secrets & SAST → SCA/Container/Sonar → Ephemeral Deploy → DAST
- Reports → Normalized JSON → AI



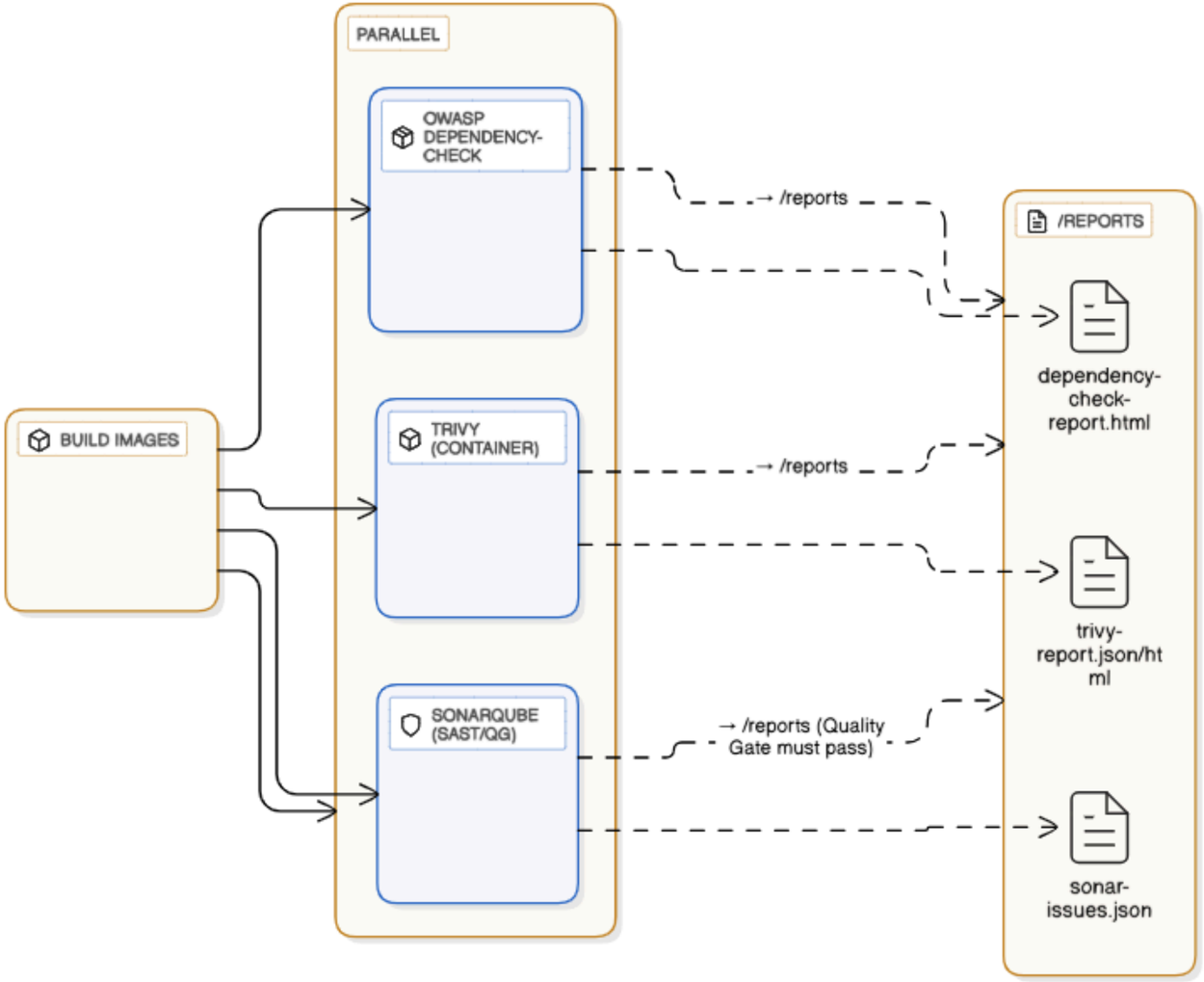
Early Scans (Parallel)

- Gitleaks — Secrets
- Semgrep — SAST (Python)



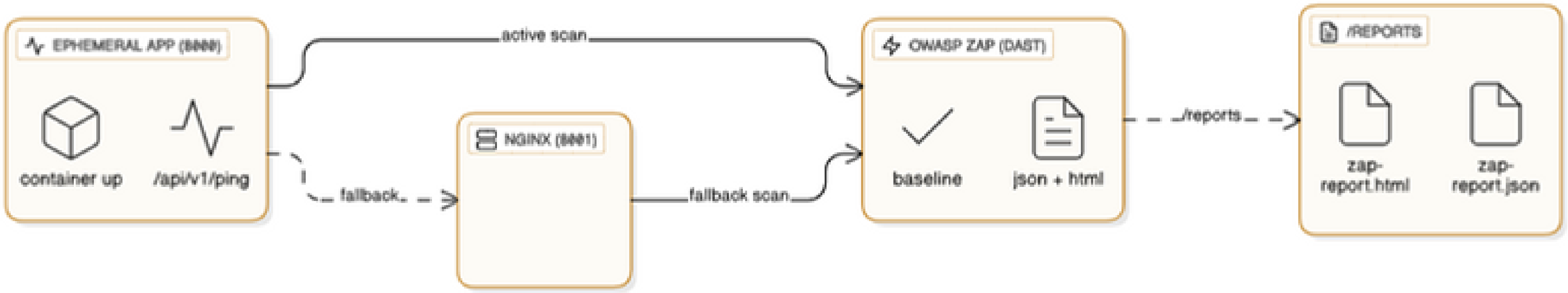
Build & Deep Scans (Parallel)

- Build images
- SCA (Dep-Check) • Container (Trivy) • SonarQube (QG)
- Reports → /reports



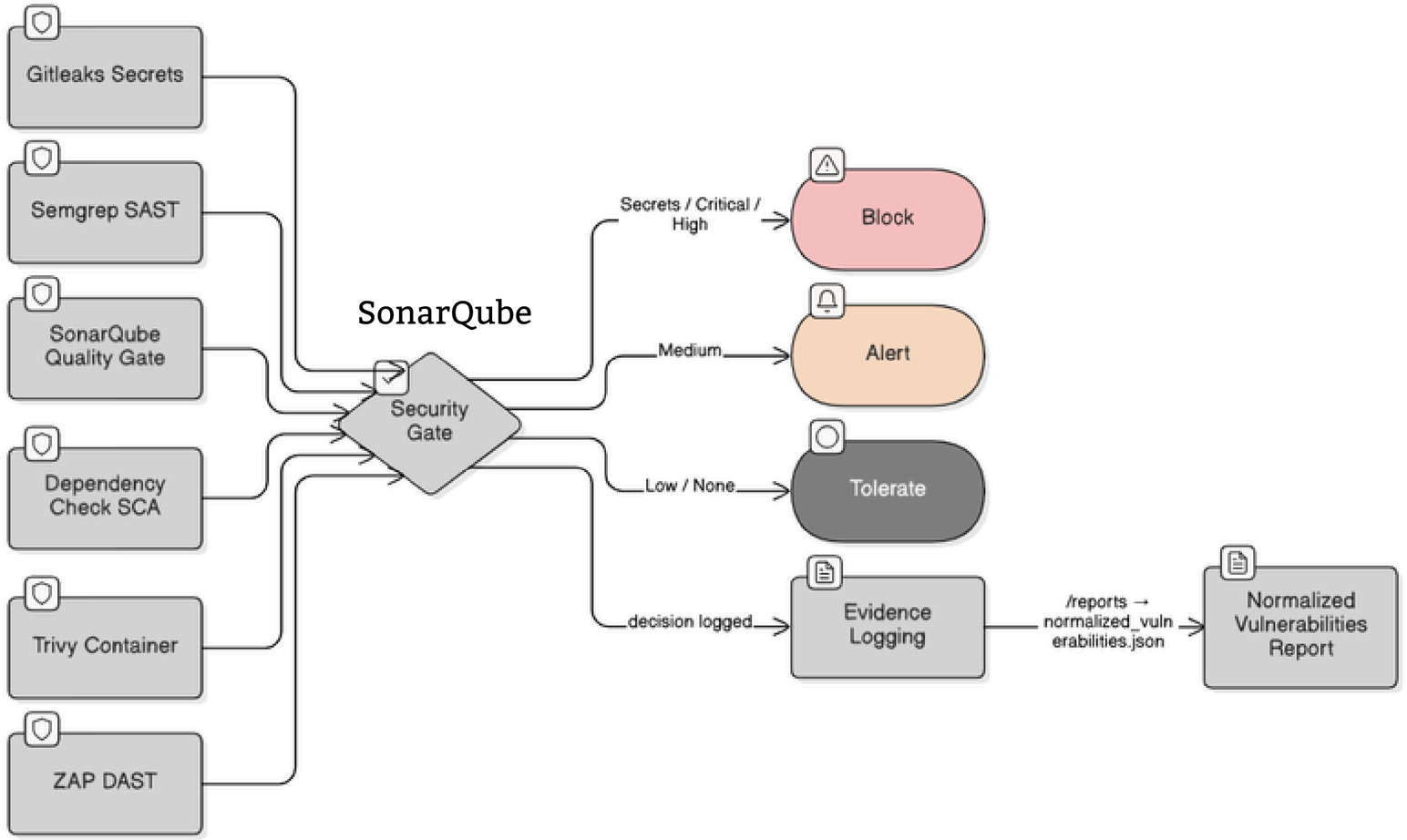
Ephemeral Deploy & DAST

- Health check (8000)
- OWASP ZAP (DAST)
- Fallback nginx (8001)
- Reports → /reports



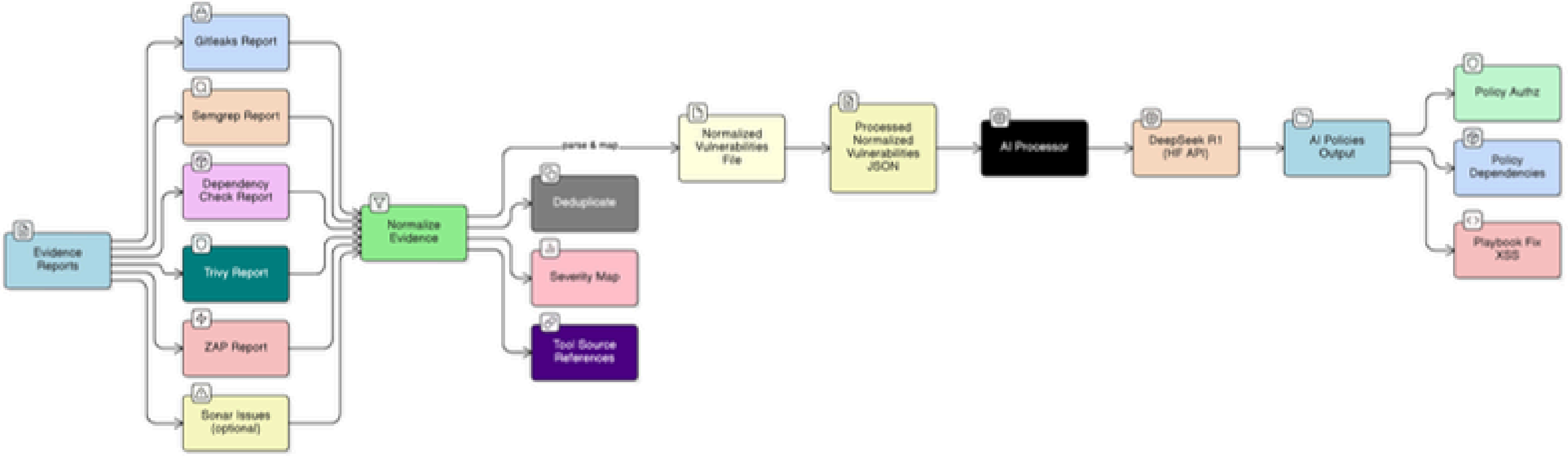
Security Gates & Outcomes

- Inputs: Secrets • SAST • SonarQube QG • SCA • Container • DAST
- Outcomes: Block / Alert / Tolerate



Normalization & Handoff to AI

- /reports → normalized JSON
- → AI policies/playbooks



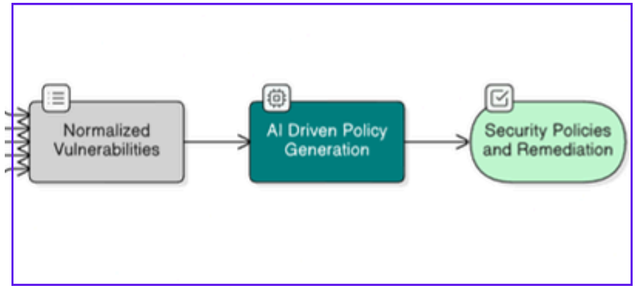
03. Normalization & Governance Bridge

Purpose

Transform diverse security tool reports into a structured, consistent format that LLMs can effectively process for policy generation

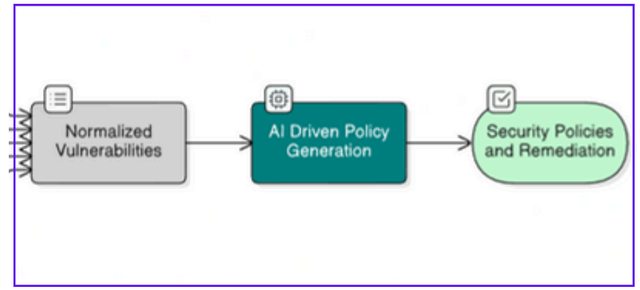
Input Sources & Formats

Tool	Report Format	Key Data Points
SonarQube	JSON	issues.type, issues.severity, issues.component, issues.message
OWASP Dependency-Check	XML	dependencies.vulnerabilities.name, dependencies.vulnerabilities.severity, dependencies.packageName
OWASP ZAP	HTML/XML	alerts.risk, alerts.name, alerts.description, alerts.solution
Trivy	JSON	Results.Vulnerabilities.VulnerabilityID, Results.Vulnerabilities.Severity, Results.Vulnerabiliti



Normalization Process

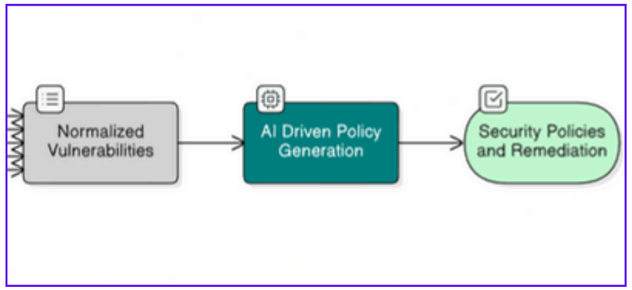
```
# Example Unified Vulnerability Schema
{
  "vulnerability_id": "CVE-2024-12345",
  "tool_source": "sonarqube",  # or "dependency-check", "zap", "trivy"
  "severity": "HIGH",          # Normalized: LOW, MEDIUM, HIGH, CRITICAL
  "category": "sql_injection", # Normalized category mapping
  "description": "Clear text vulnerability in database connection",
  "location": "app/api/database.py:45",
  "component": "postgresql-driver-2.1.0",
  "cvss_score": 7.5,
  "cwe_id": "CWE-89",
  "remediation": "Use parameterized queries",
  "timestamp": "2024-01-15T10:30:00Z"
}
```



Key Normalization Steps

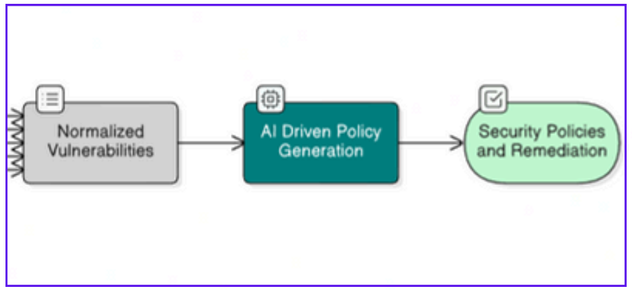
1.Severity Standardization

```
# Map tool-specific severities to unified scale
severity_mapping = {
    'sonarqube': {'BLOCKER': 'CRITICAL', 'CRITICAL': 'HIGH', 'MAJOR': 'MEDIUM'},
    'trivy': {'CRITICAL': 'CRITICAL', 'HIGH': 'HIGH', 'MEDIUM': 'MEDIUM'},
    'zap': {'High': 'HIGH', 'Medium': 'MEDIUM', 'Low': 'LOW'}
}
```



2.Category Classification

- SQL Injection → data_protection
- XSS → input_validation
- Weak Cryptography → cryptography
- Exposed Secrets → access_control



3.Deduplication & Correlation

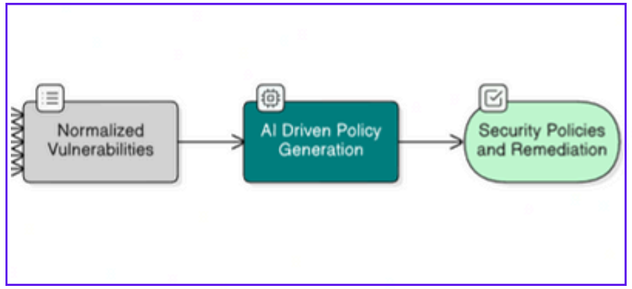
- Identify same vulnerabilities across multiple tools
- Merge duplicate findings
- Preserve tool-specific context

04. AI-Assisted Sec

AI Integration Process

LLM Models Used

Model	Parameters	Strengths	Use Case
DeepSeek R1	67B	Reasoning, Security Knowledge	Primary - Best for policy generation
LLaMA 3.3	70B	Structured Output, Coding	Comparison - Good for compliance
CodeLlama	7B	Fast, Code Security	Fallback - Quick analysis



AI Integration Process

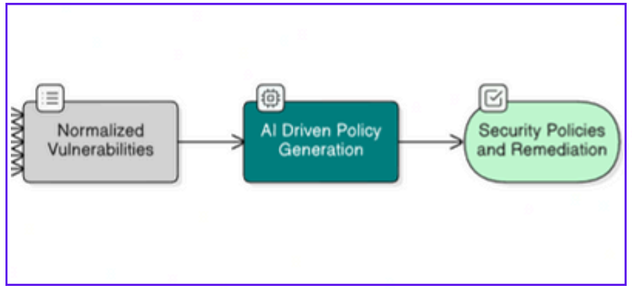
Core Script

```python
# LLM Policy Generator - Core Function
def generate_security_policy(vulnerabilities):
    """Transform vulnerabilities into NIST/ISO policies using AI"""

    # 1. Prepare data for LLM
    vuln_summary = {
        'critical': count_severity(vulnerabilities, 'CRITICAL'),
        'high': count_severity(vulnerabilities, 'HIGH'),
        'total': len(vulnerabilities)
    }

    # 2. Call DeepSeek R1 API
    policy = call_deepseek_api(vuln_summary)

    # 3. Return structured policies
    return {
        'nist_recommendations': extract_nist_controls(policy),
        'iso_controls': extract_iso_controls(policy),
        'remediation_plan': extract_remediation(policy)
    }
```



AI Integration Process

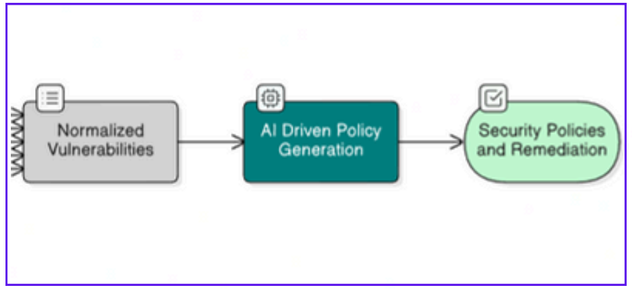
Smart Prompt Engineering

```
# Core Prompt Template
prompt = f"""
You are a cybersecurity policy expert. Generate NIST CSF/ISO 27001 policies.

VULNERABILITIES FOUND:
- Critical: {critical_count}
- High: {high_count}
- Total: {total_count}

REQUIREMENTS:
1. Create actionable security policies
2. Align with NIST CSF/ISO 27001 controls
3. Include implementation guidance
4. Specify compliance metrics

OUTPUT: Structured JSON with policies and controls
"""
```

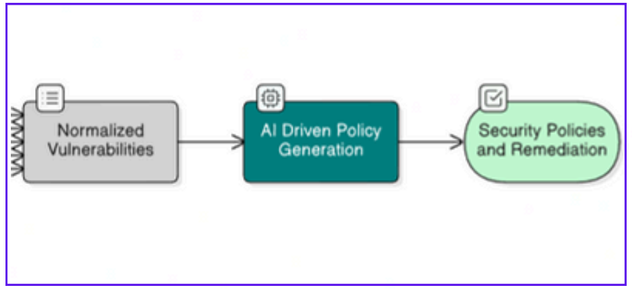


AI Integration Process

Framework-Specific Prompts

NIST CSF Prompt:

```
Focus on NIST Functions:
- PROTECT: Access control, data security
- DETECT: Security monitoring
- RESPOND: Incident response
```

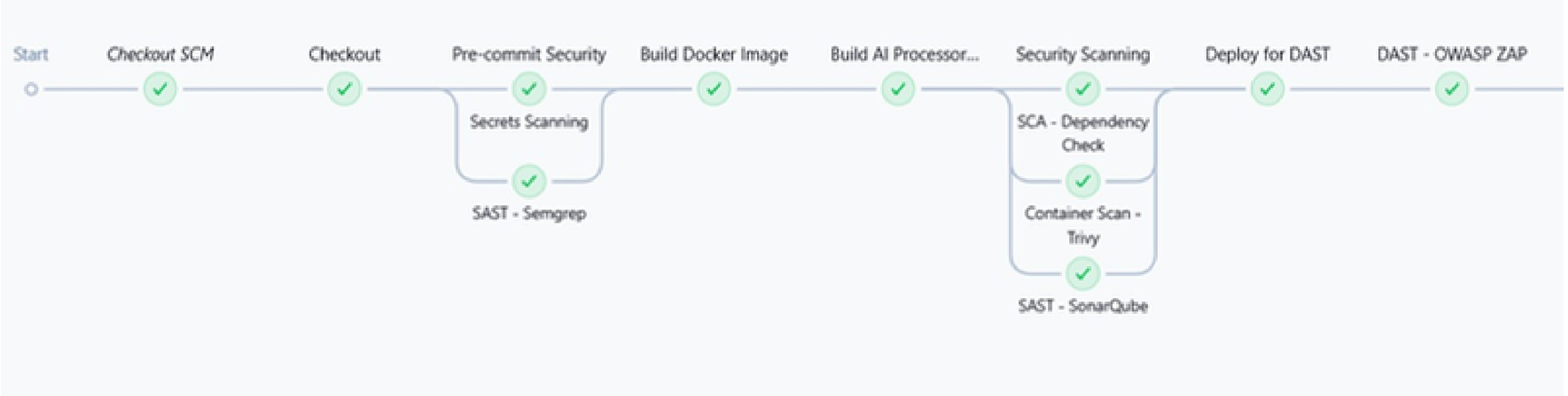


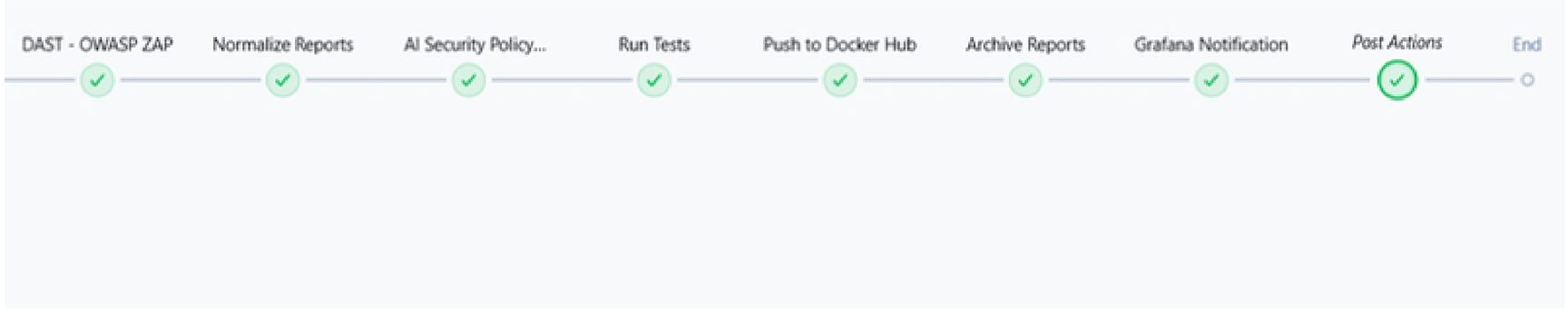
ISO 27001 Prompt:

```
Align with ISO Controls:
- A.12.6.1: Vulnerability management
- A.14.2.5: Secure development
- A.16.1.1: Incident response
```

05. Results & Demo

How does our pipeline look like on Jenkins ?





Gitleaks : gitleaks-report.json


```
{
  "scan": { "startedAt": "2025-11-07T14:02:11Z", "version": "8.18.2" },
  "findings": [
    {
      "rule": "Generic API Key",
      "file": "app/core/settings.py",
      "line": 42,
      "commit": "3ab1d5e",
      "entropy": 4.97,
      "match": "API_KEY=sk_live_**********",
      "severity": "HIGH",
      "tags": ["apikey", "secret"]
    }
  ]
}
```



```
{
  "projectInfo": { "name": "Stock Market Platform" },
  "dependencies": [
    {
      "fileName": "requests-2.31.0.tar.gz",
      "evidenceCollected": { "vendor": "python-requests" },
      "vulnerabilities": [
        {
          "name": "CVE-2023-32681",
          "severity": "HIGH",
          "cvssScore": 7.5,
          "description": "Improper certificate validation in redirects...",
          "references": [
            {"url": "https://nvd.nist.gov/vuln/detail/CVE-2023-32681"}
          ]
        }
      ]
    }
  ]
}
```


Semgrep : semgrep-report.json


```
{
  "version": "1.87.0",
  "results": [
    {
      "check_id": "python.flask.security.audit.cookie-httponly",
      "path": "app/main.py",
      "start": { "line": 88, "col": 5 },
      "end":   { "line": 88, "col": 40 },
      "extra": {
        "message": "Cookie set without HttpOnly flag",
        "severity": "HIGH",
        "metadata": { "cwe": ["CWE-1004"] }
      }
    }
  ]
}
```


OWASP Dependency-Check :
dependency-check-report.json

Trivy (Image scan) : trivy-report.json


```
{
  "ArtifactName": "saadait02/stock-market-platform:42",
  "Results": [
    {
      "Target": "python:3.11-slim",
      "Class": "os-pkgs",
      "Type": "debian",
      "Vulnerabilities": [
        {
          "VulnerabilityID": "CVE-2024-12345",
          "PkgName": "openssl",
          "InstalledVersion": "3.0.9-1",
          "FixedVersion": "3.0.12-1",
          "Severity": "CRITICAL",
          "Title": "openssl buffer overflow",
          "PrimaryURL": "https://avd.aquasec.com/nvd/cve-2024-12345"
        }
      ]
    }
  ]
}
```


OWASP ZAP (Baseline) : zap-report.json


```
{
  "site": "http://localhost:8000",
  "alerts": [
    {
      "name": "Cross-Domain Misconfiguration",
      "risk": "Medium",
      "confidence": "Medium",
      "cweid": 264,
      "desc": "CORS allows any origin",
      "instances": [
        { "uri": "http://localhost:8000/api/v1/ping", "method": "GET" }
      ],
      "solution": "Restrict allowed origins and headers."
    }
  ]
}
```


SonarQube Issues Export : sonar-issues.json


```
{
  "issues": [
    {
      "rule": "python:S4823",
      "severity": "MAJOR",
      "component": "app/core/db.py",
      "line": 27,
      "message": "Use parameterized queries to prevent SQL injection.",
      "type": "VULNERABILITY"
    }
  ],
  "paging": { "total": 1 }
}
```

ai-reports/01_Executive_Security_Summary.html

Executive Security Summary — Build #42 • commit 3ab1d5e • 2025-11-07

Quality Gate
SonarQube: PASS

Severity Totals
Critical: 1 | High: 3 | Medium: 5 | Low: 7

Gate Outcome
BLOCK (Critical present)

Top Risks (Prioritized)

Finding	Source	Severity	Fix ETA
CVE-2024-12345 (openssl) Image: python:3.11-slim • Fixed: 3.0.12-1	Trivy	Critical	24h
Cookie set without HttpOnly app/main.py:88 (Semgrep)	Semgrep	High	48h
Dependency risk: requests 2.31.0 CVE-2023-32681	OWASP Dependency-Check	High	72h

Remediation Plan

- **Container**: Rebuild base with `openssl 3.0.12-1`; pin digest; sign image (cosign).
- **SAST**: Set `HttpOnly`/`Secure` flags where cookies are created; add test coverage.
- **SCA**: Upgrade `requests` (apply constraint), regenerate SBOM, re-scan.

Traceability

All items are derived from `processed/normalized_vulnerabilities.json`. Mapping of finding → policy/playbook is available in `ai-reports/mapping.html`.

Standards Alignment

ISO 27001 A.8.7 | NIST SSDF PW.7 | OWASP ASVS 2.1

ai-reports/Policy_Authz.html (policy card for authorization/session controls)

Policy — Authorization & Session Security (Build #42)

Scope

- HTTP cookies, session handling, authentication flows in FastAPI.

Policy Rules (enforced by gate)

- Cookies must set `HttpOnly` and `Secure`. Missing flags ⇒ Severity: HIGH ⇒ **BLOCK**.
- No session identifiers in URL, logs, or client-visible storage.
- JWTs must use HS256/RS256 with rotation every 30 days; leak ⇒ **BLOCK**.
- Set `SameSite=Lax` (at minimum) for auth cookies.

Derived From

- Semgrep finding: "Cookie set without HttpOnly" @ `app/main.py:88`
- ZAP: "Cross-Domain Misconfiguration" (review CORS config)

Standards

OWASP ASVS 3.4, ISO 27001 A.8.20

ai-reports/Policy_Dependencies.html (policy card for SCA)

Policy — Dependencies (SCA) (Build #42)

Rules

- **BLOCK** if any dependency has CVSS ≥ 7.0 (Critical/High).
- Transitives are included. Dev-only packages exempt with explicit allowlist.
- SBOM must be generated for each build and archived.

Triggered Items

Package	CVE	Severity	Fix
requests 2.31.0	CVE-2023-32681	High	≥ 2.32.x
openssl (base image)	CVE-2024-12345	Critical	3.0.12-1

Sources: Dependency-Check, Trivy • Evidence: /reports/*

Standards

ISO 27001 A.8.7 • NIST SSDF PS.3.2

ai-reports/Playbook_Fix_XSS.html (playbook rendered as HTML)

Playbook — Fix Reflected XSS (DAST)

Context

ZAP flagged reflected XSS risk on `/search?q=` (Medium). The parameter is echoed unescaped.

Steps

1. In FastAPI handler, **validate** and **escape** user input before rendering.
2. Switch template engine to auto-escape (Jinja2 default) and prohibit `|safe` unless reviewed.
3. Add unit test and ZAP regression script for `/search?q=<script>`.
4. Re-run pipeline: expect ZAP to drop the alert count; update evidence.

Links

Evidence: `reports/zap-report.html` • Standard: OWASP ASVS 3.3

ai-reports/mapping.html (traceability map — findings → policy/playbook)

Traceability Mapping Build #42

Finding (normalized id)	Source Tool	Policy	Playbook	Rationale
finding:trivy:CVE-2024-12345:openssl	Trivy	Dependencies (SCA)	—	CRITICAL; fixed version available; blocks gate.
finding:semgrep:python.cookie-httponly:app/main.py:88	Semgrep	Authorization & Session	—	HIGH; cookie flags missing; aligns with ASVS 3.4.
finding:zap:reflective-xss:/search	ZAP	Authorization & Session	Fix Reflected XSS	Runtime evidence; needs input encoding and tests.

ai-reports/Appendix_Per-Domain.html

Appendix — Per-Domain Findings

Secrets

File/Line	Rule	Excerpt	Severity
app/core/settings.py:42	Generic API Key	API_KEY=sk_live_********	HIGH

SAST

Location	Issue	CWE	Severity
app/main.py:88	Cookie without HttpOnly	CWE-1004	HIGH

SCA

Package	CVE	CVSS	Severity
requests 2.31.0	CVE-2023-32681	7.5	HIGH

Container

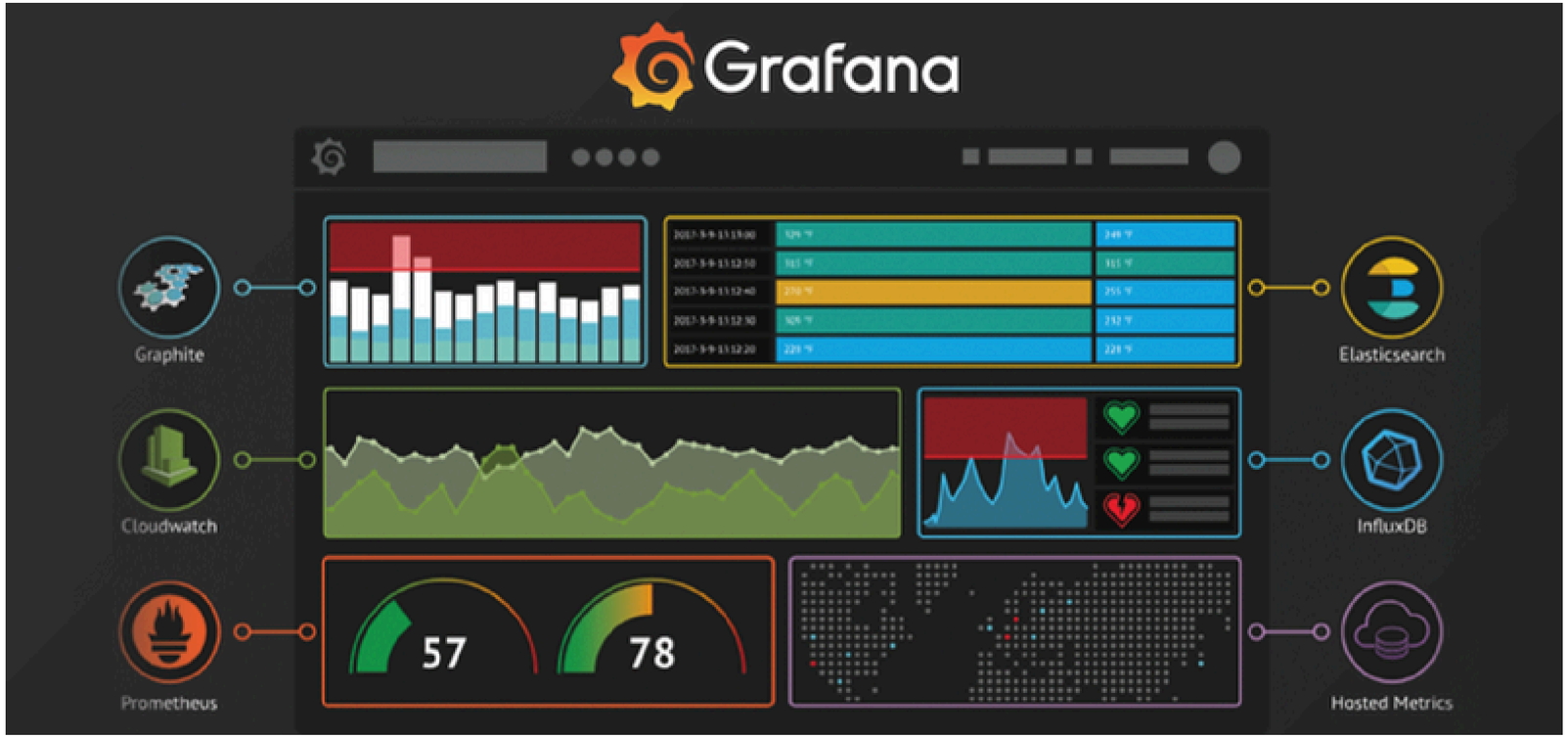
Image	Package	CVE	Severity
stock-market-platform:42	openssl	CVE-2024-12345	CRITICAL

DAST

Endpoint	Issue	Confidence	Severity
/search?q=	Reflected XSS	Medium	Medium

06. Perspectives

Grafana Visualization



azure Deployment

Thank You